

Domain Driven Design

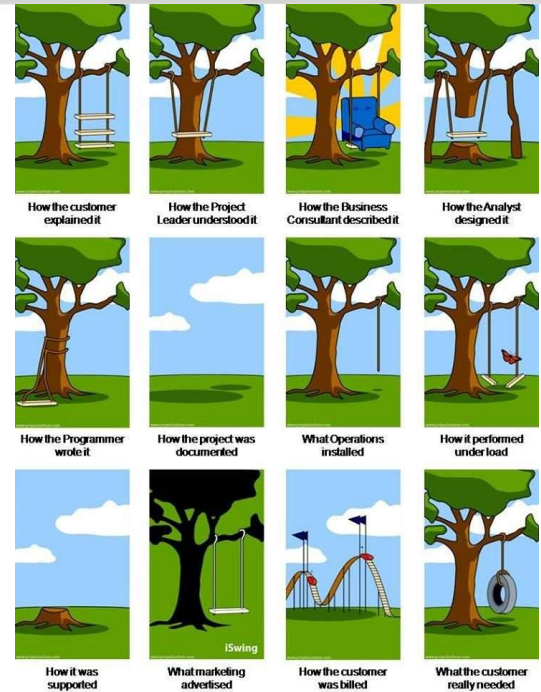
Architektur umsetzen mittels des DDD-Modells

Wir kennen jetzt die Muster, aber wissen nicht, wie wir dort hin kommen.

Es geht um das **WIE**, also wie ich zur Architektur komme.

Worum geht es

- SW ist kein Selbstzweck
- Im Fokus steht die Fachlichkeit, also die Domäne
- SW ist jedoch **nicht** das Produkt der Fachlichkeit
- SW ist das Produkt von Entwicklern, die Fachlichkeit interpretiert haben



- SW ist kein Selbstzweck
- Es geht um die Fachlichkeit, also die Domäne
- Daher ist es wichtig die Fachlichkeit verstanden zu haben
- SW ist nicht das Produkt der Fachlichkeit, sondern die Interpretation der Fachlichkeit

Was ist Domain-Driven Design (DDD)? // deutsch

<https://www.youtube.com/watch?v=KdAJz87ANs0>

Wie geht Fachlichkeit



- Entwickler sind in der Regel **nicht** die Experten
- Gemeinsame **Sprache** notwendig
- Missverständnisse >> 🐞 🐞 🐞
- Projekte dauern länger und werden teurer als erwartet

Gemeinsame Sprache

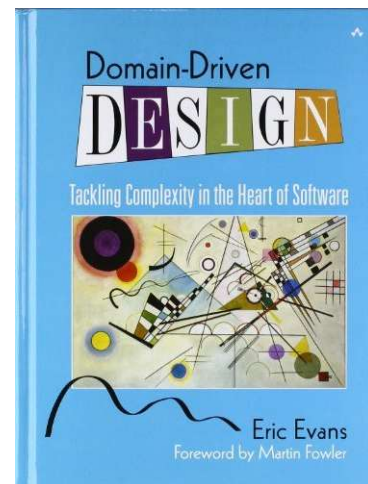
Wenn unterschiedliche Disziplinen aufeinander treffen ist es schwierig, weil jede Disziplin ihre eigene Fachjargon mit Fachbegriffen hat.

Beispiel: "**Regelmäßigkeit**" kann im juristischen Kontext etwas ganz anderes sein, wie die Formulierung das ein Tatbestand **regelmäßig** keine Haftung begründet. D. h. in diesem Kontext, dass er gemäß einer Regel keine Haftung verursacht. Hat hier also eine **andere Bedeutung als in der Umgangssprache**.

Wenn man das nicht weiß und den Begriff falsch implementiert, dann 🐞 🐞 🐞 .

Ein Weg zum gemeinsamen Verständnis

- DDD ist **ein** möglicher Lösungsansatz
- Erschien 2004 von Eric Evans
- Ansätze, um Komplexität zu beherrschen
- Werkzeuge und Leitplanken, um Domäne in ein Modell überführen zu können
- Ziel ist in interdisziplinären Teams eine klare, wohl definierte Sprache (DSL) zu schaffen



Alternativen zu DDD:

- Model Driven SW-Entwicklung (MDD) und Model Driven Architecture (MDA)
- POJO, POCO

https://de.wikipedia.org/wiki/Domain-driven_Design#Weitere_Techniken

Beispiele DSL: HTML, Logo, Regex, Matlab, SQL, Csound, Gherkin

1. Komplexität bewältigen:

1. Domain-Driven Design (DDD) bietet strukturierte Ansätze, um die Komplexität großer Softwareprojekte zu bewältigen, indem es die Domäne in überschaubare Teile unterteilt und klare Grenzen setzt.

2. Fokus auf das Wesentliche:

1. Durch den Fokus auf die Kern-Domäne und die Verwendung einer ubiquitären Sprache stellt DDD sicher, dass die Softwareentwicklung eng an den Geschäftsanforderungen ausgerichtet ist.

3. Flexibilität und Wartbarkeit:

1. Die klare Trennung von Verantwortlichkeiten und die Verwendung von Bounded Contexts führen zu einer flexibleren und wartbareren Architektur, die leichter an sich ändernde Anforderungen angepasst werden kann.

4. Kohärenz und Konsistenz:

1. Die Verwendung eines gemeinsamen Domänenmodells und einer ubiquitären Sprache fördert die Kohärenz und Konsistenz innerhalb des Entwicklungsteams und zwischen den Entwicklern und den Domänenexperten.

1.Langfristiger Erfolg:

1. DDD hilft, langfristig erfolgreiche Softwarelösungen zu entwickeln, die skalierbar, wartbar und anpassungsfähig sind, indem es die Grundlagen für eine solide und nachhaltige Architektur legt.

Zusammengefasst ist "Domain-Driven Design" von Eric Evans ein essenzielles Werk für jeden Softwarearchitekten und -entwickler, das wertvolle Prinzipien und Praktiken bietet, um komplexe Softwareprojekte erfolgreich zu bewältigen.

Zentrale Aspekte des Buches auf DDD

Fokus auf die Domäne

- Tiefes Verständnis der Geschäftsdomäne
- Erstellung eines präzisen Domänenmodells

Ubiquitous Language

- Gemeinsame Sprache zwischen Entwicklern und Domänenexperten
- Sicherstellung, dass alle Beteiligten die gleichen Begriffe und Konzepte verwenden

Bounded Contexts

- Unterteilung großer Domänen in kleinere, überschaubare Teile
- Jeder Kontext hat ein eigenes Modell und eine eigene Sprache

Zentrale Aspekte des Buches:

1. Fokus auf die Domäne:

1. **Domänenmodell:** Das Buch betont die Bedeutung eines tiefen Verständnisses der Geschäftsdomäne und die Erstellung eines präzisen Domänenmodells, das die zentralen Konzepte und Beziehungen der Domäne widerspiegelt.
2. **Ubiquitous Language:** Eine gemeinsame Sprache zwischen Entwicklern und Domänenexperten, die sicherstellt, dass alle Beteiligten die gleichen Begriffe und Konzepte verwenden.

2. Bounded Contexts:

1. **Kontextgrenzen:** Das Konzept der Bounded Contexts hilft, große und komplexe Domänen in kleinere, überschaubare Teile zu unterteilen, die jeweils ein eigenes Modell und eine eigene Sprache haben.
2. **Kontextübergreifende Kommunikation:** Definiert, wie verschiedene Bounded Contexts miteinander interagieren, um eine kohärente Gesamtlösung zu gewährleisten.

Zentrale Aspekte des Buches auf DDD

Entities, Value Objects, Aggregates

- Entities: Objekte mit eindeutiger Identität
- Value Objects: Objekte ohne eindeutige Identität, definiert durch Attribute
- Aggregates: Konsistente Einheiten aus Entities und Value Objects

Repositories und Factories

- Repositories: Vereinfachen den Zugriff auf Aggregates
- Factories: Mechanismen zur Erstellung komplexer Objekte

Anti-Corruption Layer

- Schutz der Kern-Domäne vor äußeren Einflüssen
- Sicherstellung der Integrität und Konsistenz des Domänenmodells

Zentrale Aspekte des Buches:

1.Entities, Value Objects und Aggregates:

1. **Entities:** Objekte, die eine eindeutige Identität haben und über die Zeit hinweg verfolgt werden.
2. **Value Objects:** Objekte, die keine eindeutige Identität haben und durch ihre Attribute definiert sind.
3. **Aggregates:** Gruppen von Entities und Value Objects, die zusammen eine konsistente Einheit bilden und als Ganzes verwaltet werden.

2.Repositories und Factories:

1. **Repositories:** Abstraktionen, die den Zugriff auf Aggregates vereinfachen und die Persistenzschicht kapseln.
2. **Factories:** Mechanismen zur Erstellung komplexer Objekte und Aggregates, die die Komplexität der Objekterstellung kapseln.

3.Anti-Corruption Layer:

1. **Schutz vor äußeren Einflüssen:** Ein Schutzschild, der die Kern-Domäne vor den Auswirkungen schlechter Designs oder ungeeigneter Technologien in anderen Teilen des Systems schützt.

Datenpersistenz

CRUD, CQRS und Event-Sourcing

Worum geht es

Datenhaltung ist **unabhängig** vom
gewählten Architekturmuster

Beispiel: Mikromobilitäts-Verleih-Service

- Neues Fahrrad: Datensatz aufnehmen
- Bestand prüfen: Daten lesen
- Status setzen: Datenfeld aktualisieren
- Fahrrad kaputt: Datensatz entfernen

Sehr logisch, deshalb:

Create**R**ead**U**ppdate**D**elete



<https://www.youtube.com/watch?v=ss9wnixCGRY>

Beispiel Vermietung von Transportmitteln (Mikromobilität*)

=====

- Ein Transportmittel pro Datensatz
- Wird angelegt wenn Transportmittel aufgenommen wird
- Beim Ausleihen wird Datensatz aktualisiert
- Wenn Transportmittel kaputt wird Datensatz entfernt

Sehr logisch, weil wir das so gelernt haben!

Wie wir mit Daten umgehen, ist aus konzeptioneller Sicht immer gleich.

**„Mikromobilität ist die Fortbewegung mit elektrisch motorisierten sowie nicht motorisierten Kleinst- und Leichtfahrzeugen, auch Elektrokleinstfahrzeuge genannt. Dazu zählen E-Tretroller bzw. E-Scooter, Tretroller, Segways, E-Leichtfahrzeuge, Hoverboards, Monowheels und auch E-Skateboards und klassische Skateboards.“*

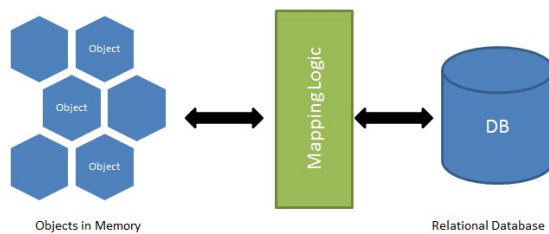
Datenbank für Verleih-Service

Relationale Datenbank

- MS SQL, MySQL, Oracle, PostgreSQL, ...

O/R Mapping

- Entity Framework, Dapper, NHibernate, ...



Frage an TN, ob O/R Mapper und EF Core bekannt

Web API für Verleih-Service

Einheitliche Schnittstelle für Clients mit Web API



Desktop



Web



Mobile

REST-Operationen mittels HTTP-Verben

HTTP-Verben (Methoden)	CRUD-Operationen
GET	Read
POST	Create
PUT / PATCH	Update / Partial Update
DELETE	Delete



Frage an TN, ob Rest API bekannt

Diskussion: **Ist CRUD sinnvoll und gut?**

CRUD hat sich etabliert.

ABER: Wo Licht ist, gibt es auch Schatten.

Es gibt Dinge, die mit CRUD nicht so gut harmonieren.

Welche?

- Löschen meistens keine gute Idee: Soft-Delete mittels Flag.
- Selbst ein Trivialbeispiel kann ziemlich komplex werden
- Insbesondere bei großen Datenmodellen

Dann kommt die Fachabteilung:

Wir hätten gerne Reports, wie oft Fahrräder trotz zweifacher Verlängerung verspätet zurück gegeben werden.

- > Resultat:
- DB-Schema anpassen
 - Code anpassen und hoffen, dass nichts kaputt geht
 - Code für Report schreiben
 - Auf neue Daten warten
 - Reports an Fachabteilung zurück geben

--> Langsamer Entwicklungszyklus

--> Evtl. Nabesserung in neuer Iteration notwendig weil mangelhafte Implementierung

Wenn die Änderung nicht passt, werden Änderung i.d.R. nicht rückgängig gemacht

--> verwaiste Datenstrukturen >> Datenschema wächst weiter >> Komplexität >>
Technical Depth

Event-Sourcing

The single source of truth

Anderes Beispiel: Girokonto

- Bank speichert **niemals** den Saldo (Kontostand)
- Nachvollziehbarkeit der Transaktionen
- Schlechte Idee
 - **Delete:** Ohnehin eine blöde Idee
 - **Update:** Buchungen im Nachhinein ändern?
- Also nur Create und Read
 - Status Quo wird **nicht** gespeichert
 - Sondern nur die **Änderungen** zum Status quo
- Kompensation mit Gegentransaktion statt Update



Beispiel Girokonto

- Bank speichert **nicht** den Kontostand (Saldo)

Warum ist am Ende des Geldes noch so viel Monat da?

- Nachvollziehbarkeit der Transaktionen
- Es gibt kein Delete, kein Update
- D. h. nur Create und Read
- Das ist wichtig für Vertrauensbindung zum Kunden

Wir speichern Änderungen statt den Status Quo.

Und wenn mal was schief geht?

- > Gegentransaktion, um Fehler zu kompensieren
- Mit den Rohdaten können wir die Vergangenheit neu interpretieren

Beispiele

- Wieviel % des Gehalts geht für Miete weg?
- Lohnt sich Lotto spielen?
- Wie oft im Monat gehst du essen?
- Bist du sparsam oder verschwenderisch?

Verleih mit Transaktionen statt CRUD!

Funktioniert auch für **Verleih-Service!**

>> Event-Sourcing <<

Darüber hinaus lassen sich Fragen beantworten:

- Wie oft wird ein bestimmtes Transportmittel ausgeliehen?
- Wie oft wurde es verlängert?
- Was sind Top 10 der beliebtesten Transportmittel?

Statt nur den **aktuellen Zustand** zu persistieren, fokussieren wir uns auf **Änderungen** über die Zeit hinweg.



Damit verhindern wir, dass so ein Müll herauskommt:

- Der böse Wolf aktualisiert das IsDeleted Flag vom Rotkäppchen
- Weil er es nicht löschen darf, sonst kann der Jäger kein Undelete machen.

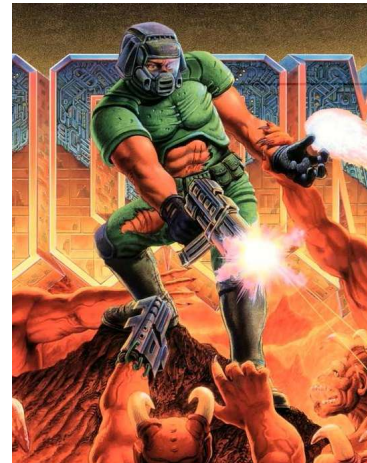
Einzelne Einträge in einer wachsenden Liste

- AppendOnlyList<T>
- Immer in Vergangenheitsform

Vorteile von Event-Sourcing

- Historische Analysierbarkeit von Daten
- Event-Log for free!
- Replays mit What-if Analysen
- Replays mit simulierten Events
- Verbesserte Fehlersuche und Debugging
- Flexible Darstellungsformen möglich

Kritik: Speicher, Performance, DSGVO



- Event-Log for free: Praktisch wenn Audit-Log eine Vorgabe ist

Trivia: Doom arbeitet unter der Haube mit Event-Sourcing, (den Begriff gab es damals noch nicht) um den Spielzustand effizient zu verwalten und zu reproduzieren. Es vereinfachte neben dem Debugging auch die Implementierung des Netzwerkcodes für Multiplayer.

- Flexible Darstellungsform mit Lesemodellen, die kontinuierlich aktualisiert werden
- Theoretisch muss es nicht mal persistiert werden, da Tabelle neu berechnet werden kann

Nachteile / Kritik

- Speicherverbrauch
 - evtl. Filterung notwendig oder Archivierung
 - Bsp. Abtastrate bei Sensoren
 - Speicher kostet nicht mehr viel
- Performance bei Replays
 - Vorbeugen mit Snapshots / Cache
- DSGVO Relevanz: Recht auf "Vergessen werden"